VAPT Report

Task – Mobile Penetration Testing

Title-Mobile Application Security Assessment Report

Application Name:InsecureBankv2.apk

Submitted by :-
R.Yoganandh
Date:05-03-2026

Introduction:

The objective of this assessment was to evaluate the security posture of the Android application **InsecureBankv2**. The testing process involved both **static analysis** and **dynamic analysis** techniques to identify potential security vulnerabilities that could impact the confidentiality, integrity, and availability of the application and the sensitive data it processes.

During the assessment, the application package (APK) was analyzed to review its internal structure, configuration files, and source code in order to detect security weaknesses. Additionally, runtime testing was performed by interacting with the application in a controlled environment to observe its behavior and identify vulnerabilities related to authentication, data storage, and application communication.

The assessment identified several security issues that could potentially be exploited by attackers. These included **insecure data storage mechanisms, exposed broadcast receivers, hardcoded credentials, and weaknesses in authentication logic**. If exploited, these vulnerabilities could allow unauthorized access to sensitive information, manipulation of application functionality, or compromise of user accounts.

Addressing these vulnerabilities and implementing secure development practices would significantly improve the overall security posture of the application and reduce the risk of potential exploitation.

Scope:

The scope of this assessment included the security evaluation of the Android mobile application InsecureBankv2. The testing focused on identifying potential security vulnerabilities within the application through both static and dynamic analysis techniques. The assessment covered the analysis of the application package (APK), source code review, configuration and manifest analysis, authentication mechanisms, data storage practices, and communication with backend services. Testing was conducted in a controlled environment using an Android emulator to simulate real-world usage scenarios and identify weaknesses that could be exploited by attackers. The scope was limited to the mobile application itself and did not include testing of external infrastructure, backend servers, or third-party services associated with the application.

Setup:

Lab Testing Environment:

Required Tools:

1. ADB - Used to interact with the Android device and install the application
2. APKTool- Used to decompile the APK and analyze its internal structure
3. JADX- Intended for converting bytecode into readable Java source code.
4. Kali linux
5. Burp Suite – For intercepting and modifying the web traffic

Device:

1.Google pixel 3 (pie 9.0)

In Genymotion –

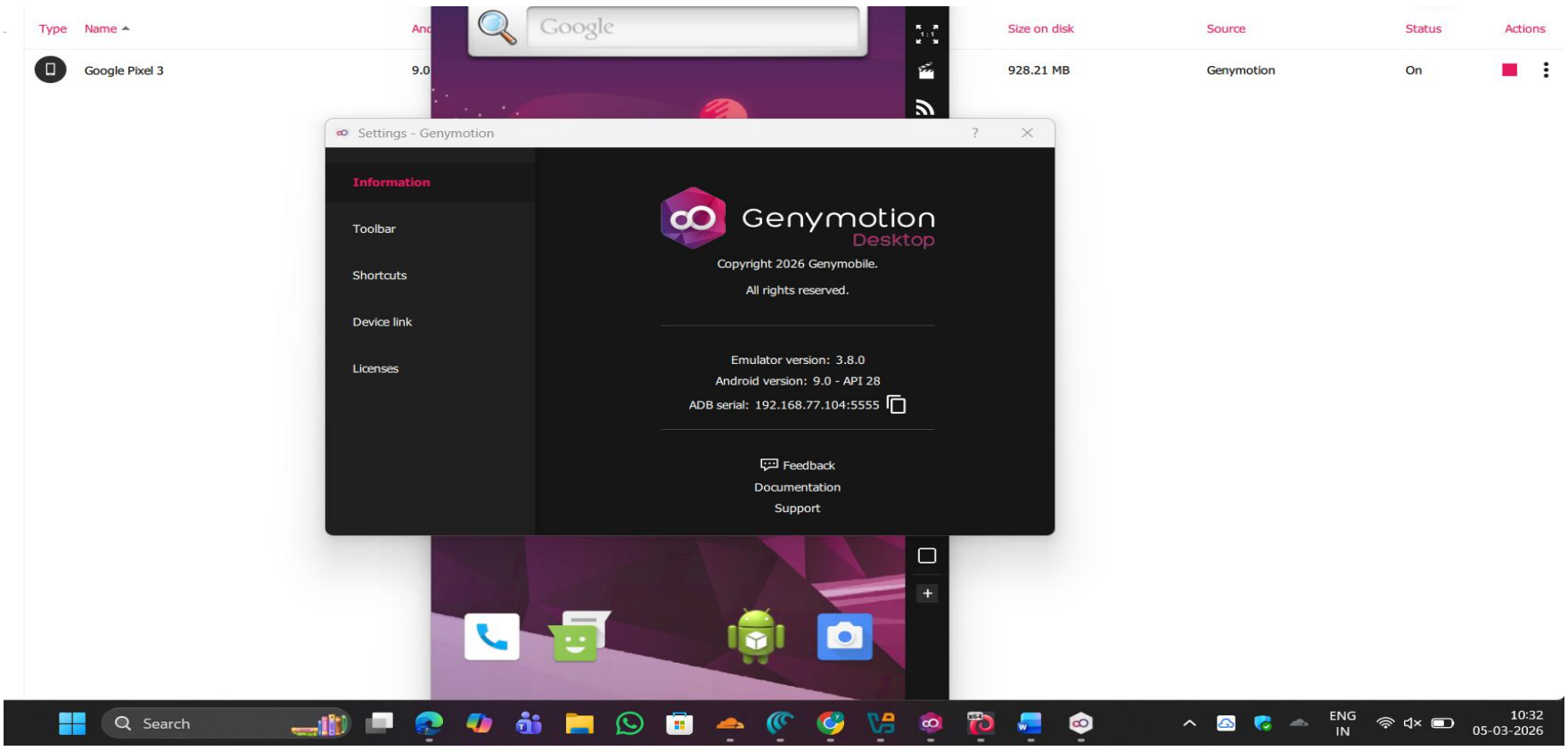

Image 1

ADB :-

```
┌──(yogi㉿kali)-[~]
└─$ adb connect 192.168.77.104:5555
* daemon not running; starting now at tcp:5037
* daemon started successfully
connected to 192.168.77.104:5555

┌──(yogi㉿kali)-[~]
└─$ adb devices
List of devices attached
192.168.77.104:5555     device
```

Image 2

2.Install the APK:-

```
┌──(yogi㉿kali)-[~/InsecureBankv2]
└─$  git clone https://github.com/dineshshetty/Android-InsecureBankv2.git
Cloning into 'Android-InsecureBankv2'...
remote: Enumerating objects: 1791, done.
remote: Counting objects: 100% (189/189), done.
remote: Compressing objects: 100% (36/36), done.
remote: Total 1791 (delta 160), reused 153 (delta 153), pack-reused 1602 (from 1)
Receiving objects: 100% (1791/1791), 61.07 MiB | 2.51 MiB/s, done.
Resolving deltas: 100% (625/625), done.
```

Image 3

- First install the application - The application was installed on a testing device using the Android Debug Bridge.

```
┌──(yogi㉿kali)-[~/InsecureBankv2/Android-InsecureBankv2]
└─$ adb devices
List of devices attached
192.168.77.104:5555     device


┌──(yogi㉿kali)-[~/InsecureBankv2/Android-InsecureBankv2]
└─$ adb install InsecureBankv2.apk
Performing Streamed Install
Success
```

Image 4

- Launch the apk to the android –

```
┌──(yogi㉿kali)-[~/InsecureBankv2/Android-InsecureBankv2]
└─$ adb shell monkey -p com.android.insecurebankv2 1
  bash arg: -p
  bash arg: com.android.insecurebankv2
  bash arg: 1
args: [-p, com.android.insecurebankv2, 1]
 arg: "-p"
 arg: "com.android.insecurebankv2"
 arg: "1"
data="com.android.insecurebankv2"
Events injected: 1
## Network stats: elapsed time=148ms (0ms mobile, 0ms wifi, 148ms not connected)
```

Image 5

- After connecting to the android device, Perform Static Analysis and analyze the APK without running it.
- To analyze the internal structure of the application, the APK file was decompiled using **APKTool**.

```
┌──(yogi㉿kali)-[~/Android-InsecureBankv2]
└─$ apktool d InsecureBankv2.apk
I: Using Apktool 2.7.0-dirty on InsecureBankv2.apk
I: Loading resource table...
I: Decoding AndroidManifest.xml with resources...
I: Loading resource table from file: /home/yogi/.local/share/apktool/framework/1.apk
I: Regular manifest package...
I: Decoding file-resources...
I: Decoding values */* XMLs...
I: Baksmaling classes.dex...
I: Copying assets and libs...
I: Copying unknown files...
I: Copying original files...

┌──(yogi㉿kali)-[~/Android-InsecureBankv2]
└─$ ls
 AndroLabServer   InsecureBankv2.apk   README.markdown   Thumbs.db          Walkthroughs
 InsecureBankv2   LICENSE              Spoilers         'Usage Guide.pdf'   wip-attackercode

┌──(yogi㉿kali)-[~/Android-InsecureBankv2]
└─$ cd InsecureBankv2

┌──(yogi㉿kali)-[~/Android-InsecureBankv2/InsecureBankv2]
└─$ ls
AndroidManifest.xml  apktool.yml  original  res  smali

┌──(yogi㉿kali)-[~/Android-InsecureBankv2/InsecureBankv2]
```

Image 6

The decompilation process extracted several components including:

- AndroidManifest.xml
- res directory (resources)
- smali code

These files allow us to review application configuration and logic without executing the application.

1.Vulnerability: Exported Activity (Authentication Bypass)

Analyze AndroidManifest:- Severity: High

Description:

The AndroidManifest.xml file defines the application's configuration, permissions, and exposed components.

We can review this file to identify:

- Application permissions
- Exported activities
- Services and broadcast receivers
- Backup configuration
- Debugging Enabled

Misconfigurations in this file may lead to unauthorized access or data exposure.

```
    <uses-feature android:glEsVersion="0x00020000" android:required="true"/>
    <application android:allowBackup="true" android:debuggable="true" android:icon="@mipmap/ic_launcher" android:label="@string/app_name" android:theme="@andr
oid:style/Theme.Holo.Light.DarkActionBar">
        <activity android:label="@string/app_name" android:name="com.android.insecurebankv2.LoginActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
        <activity android:label="@string/title_activity_file_pref" android:name="com.android.insecurebankv2.FilePrefActivity" android:windowSoftInputMode="adj
ustNothing|stateVisible"/>
        <activity android:label="@string/title_activity_do_login" android:name="com.android.insecurebankv2.DoLogin"/>
        <activity android:exported="true" android:label="@string/title_activity_post_login" android:name="com.android.insecurebankv2.PostLogin"/>
        <activity android:label="@string/title_activity_wrong_login" android:name="com.android.insecurebankv2.WrongLogin"/>
        <activity android:exported="true" android:label="@string/title_activity_do_transfer" android:name="com.android.insecurebankv2.DoTransfer"/>
        <activity android:exported="true" android:label="@string/title_activity_view_statement" android:name="com.android.insecurebankv2.ViewStatement"/>
        <provider android:authorities="com.android.insecurebankv2.TrackUserContentProvider" android:exported="true" android:name="com.android.insecurebankv2.T
rackUserContentProvider"/>
        <receiver android:exported="true" android:name="com.android.insecurebankv2.MyBroadCastReceiver">
            <intent-filter>
                <action android:name="theBroadcast"/>
            </intent-filter>
        </receiver>
        <activity android:exported="true" android:label="@string/title_activity_change_password" android:name="com.android.insecurebankv2.ChangePassword"/>
        <activity android:configChanges="keyboard|keyboardHidden|orientation|screenLayout|screenSize|smallestScreenSize|uiMode" android:name="com.google.andro
```


Image 7

Step 1 — Identify Exported Activity using cmd : “**nano AndroidManifest.xml”**

- Example findings :

  <activity
  android:name=".PostLogin"
  android:exported="true"/>

- This means the activity can be launched **without logging in**.

Step 2 — Exploit Using ADB

- First check package name and launch activity directly

```
┌──(yogi㉿kali)-[~/Android-InsecureBankv2]
└─$ adb shell pm list packages | grep bank
package:com.android.insecurebankv2

┌──(yogi㉿kali)-[~/Android-InsecureBankv2]
└─$ adb shell am start -n com.android.insecurebankv2/.PostLogin
Starting: Intent { cmp=com.android.insecurebankv2/.PostLogin }
```

Image 8

- If the dashboard opens → **authentication bypass vulnerability**

Impact:

- Unauthorized users can access restricted functionality.

Recommendation:

- Set sensitive activities as **android:exported="false"** and implement proper authentication checks to prevent unauthorized access.

2.Vulnerability: Insecure Storage of Credentials

Severity: High

Description:

Step 1: Open Source Code

Open Important Classes**,**Inside the package open these files:

- com.android.insecurebankv2
- DoLogin-
  Search it by using ctrl +f
  MyBroadcastReceiver
  DoLogin
- UserFunctions
- DBHelper
- MyBroadcastReceiver
  “MyBroadcastReceiver”

Above mentioned are the **main vulnerable components**.

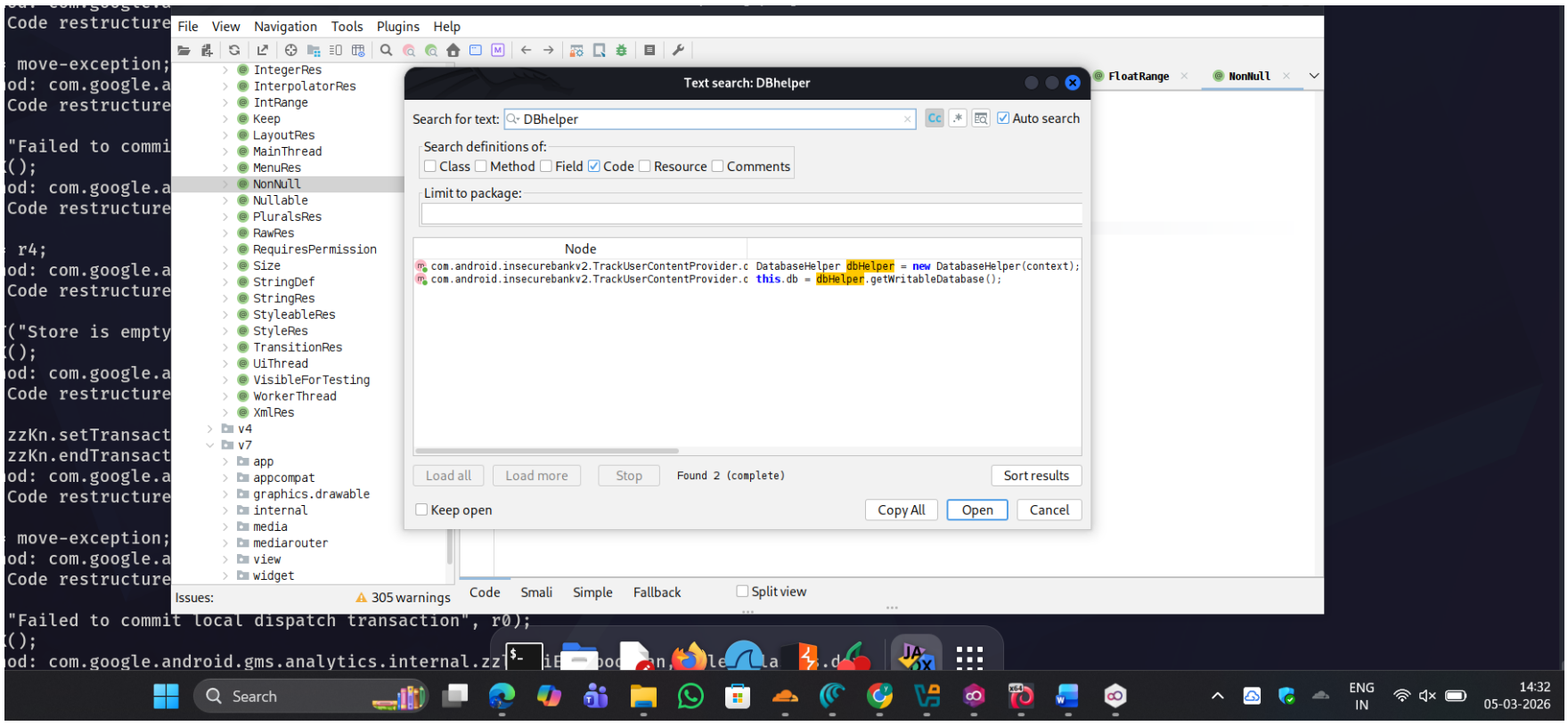

Image 9

Impact:

- An attacker with access to the device can extract stored credentials.

Recommendation:

- Sensitive credentials should be stored using Android Keystore or encrypted securely.

3.Vulnerability: Hardcoded Credentials

Severity: High

Description:

Step 1 — Open Source Code

- Search inside JADX: “Ctrl + Shift + F”
- Search for the password & admin
- By checking the below Files in the JADX:

  DoLogin.java
  UserFunctions.java
  DBHelper.java

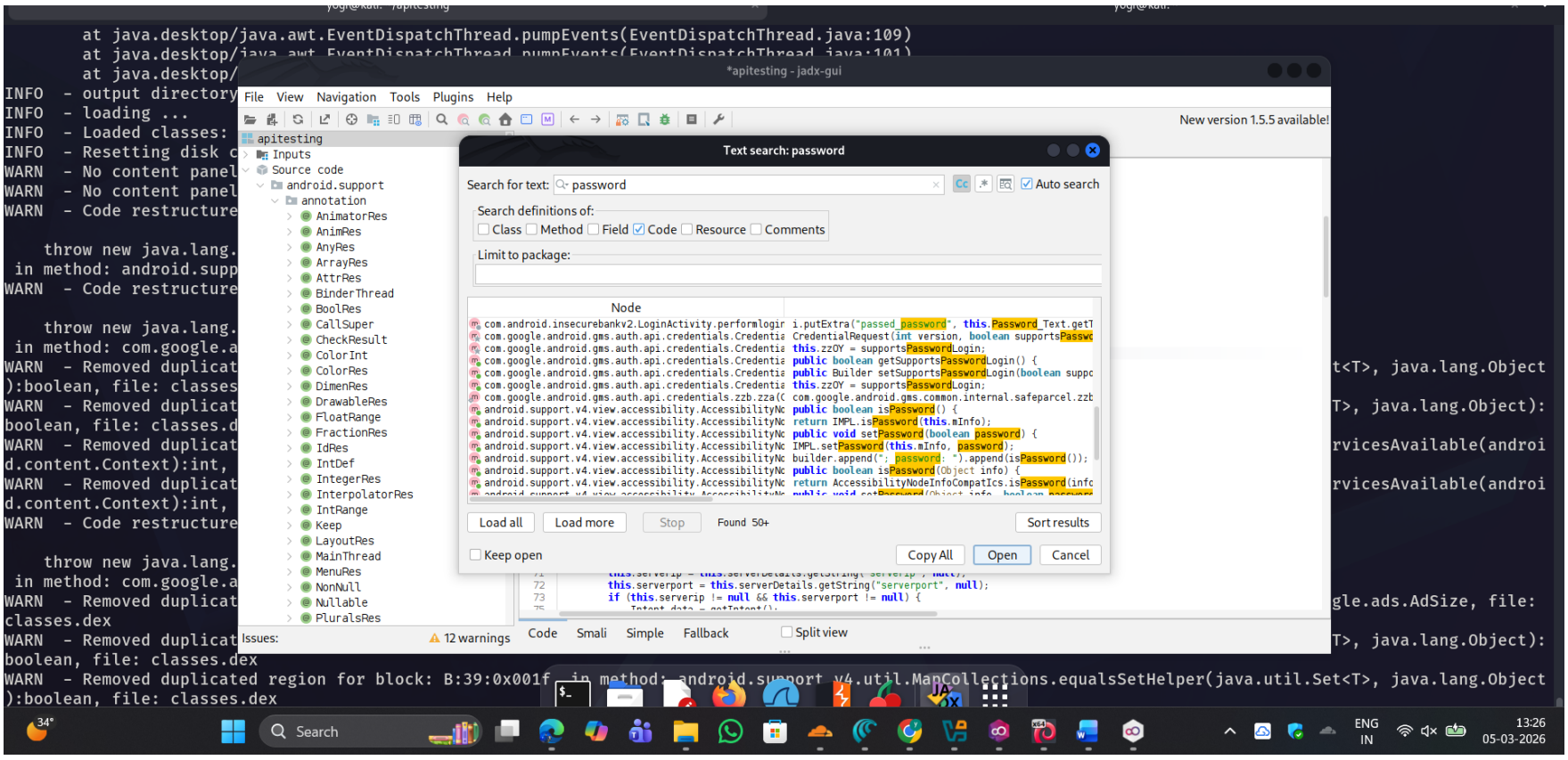

Image 10

Impact:

Attackers can reverse engineer the APK using **JADX** and discover credentials or authentication mechanisms.This allows attackers to Understand login logic,Bypass authentication, Access protected resources

Recommendation:

Sensitive credentials should never be embedded in application code. Authentication logic should be handled securely on the server side.

Conclusion:

The security assessment of the Android application InsecureBankv2 identified several vulnerabilities that could potentially impact the security of the application and its users. Through static and dynamic analysis, multiple issues such as insecure data storage, hardcoded credentials, exposed broadcast receivers, and authentication weaknesses were discovered.

If exploited, these vulnerabilities could allow attackers to gain unauthorized access, retrieve sensitive information, or manipulate application functionality. Addressing these security issues and implementing secure coding practices would significantly improve the overall security posture of the application. It is recommended that the development team review the identified findings and apply appropriate remediation measures to reduce potential security risks.

References:

- OWASP Mobile Security Testing Guide
- OWASP Mobile Top 10 Security Risks

- Android Developers Security Best Practices Documentation
- JADX Decompiler Documentation